Jasmine Kaur
IDS 568

Performance Report

Introduction

For this assignment, I had built a LLM inference API using FastAPI with caching and batching so that it can improve the performance in different type of workloads. The goal of this whole thing was to create the type of system that is able to take care and handle multiple requests efficiently instead of processing each one individually one request at a time

Compute Pathways

So in the basic setup, each of the requests were being individually processed which is not the best as it's going to slow down the performance if there's too many requests. With batching, multiple requests are going to be grouped together and processed at the same time which is going to help a lot as it is going to reduce the repeated calls. Also, caching is used so if the same prompt is sent again, then the system will be able to return the stored response instead of rerunning the model. This is very helpful as it reduces any unnecessary computation.

Benchmark Methodology

So I wanted to test the performance and I was able to successfully do that by running the benchmarks at different levels or request sizes and concurrency levels. I had tested concurrency levels of 1,2, and 5 as well as request sizes of 5,10, and 20 where I was able to find out from the results of how the system is able to handle high and low load conditions. Different metrics, like the total execution time and the average latency were also recorded and the raw results were saved in the benchmarks folder.

Jasmine Kaur
IDS 568

Latency and Throughput Results

The results indicated the increasing concurrency improved the overall performance. When the concurrency level was set to 1 then the system was processing all the requests more sequentially which in return increased the total time. Once the concurrency levels were increasing to 2 and 5, then multiple requests were handled at the same time which reduced the total execution time and improved the throughput.

Also, the average latency stayed stable across most of the tests that were done. Even though latency was increasing at higher loads by a little, the system was able to handle the requests efficiently. This indicates that the system is able to scale good as the number of requests increase.

Cache Behavior

Whenever the prompts are repeated then the caching helps improve the performance of the system. In cold cache citations then the system has to generate all of the responses from scratch. In the warm-cache citations, the repeated prompts are faster due to the fact the responses are already saved and stored. This helps a lot by improving efficiency and reducing the processing time.

Resource Usage

Okay so there is a trade off between the resource usage and the performance. Caching helps reduce the computation but it needs more memory to store the previous last results while batching helps improve efficiency but it can cause an increase in the waiting time for individual requests. So due to this batching and caching have to have their settings changed based on the system being used.

Jasmine Kaur
IDS 568

Trade-Off Analysis

There are are trade-offs of how the caching and batching are configured. Caching helps with the speed for repeated inputs but it can return the outdated responses if it is not being managed properly. When it comes to batches the larger sizes are able to improve the throughput but it has increase the latency if the time period of waiting for processing the requests is too long. That is why it is highly important to make sure to find the right balance for having good overall performance.

Scaling Strategy

So to make the system even better and improve it then having more advanced caching systems could be helpful and having better monitoring of the system resources. So in a real-world system, tracking cache hit rates and optimizing batch sizes would really aid with improving the performance. Overall, the current system showcases that batching and caching can make the API more efficient and scalable.

Jasmine Kaur
IDS 568